

SISTEMAS OPERACIONAIS – 2026/II

Contagem Paralela de Objetos em Matriz Binaria

Conectividade 8 com Pthreads e Union-Find

Elisa Ely de Oliveira Rigotti | Maria Julia Escobar Correia de Melo

PUCRS – Escola Politecnica | Prof. Filipo Novo Mor

ROTEIRO

Agenda

01 Definicao do problema

02 Solucao sequencial

03 Arquitetura paralela

04 Validacao e testes

05 Resultados de desempenho

06 Conclusoes

PROBLEMA

Contagem de componentes conexos

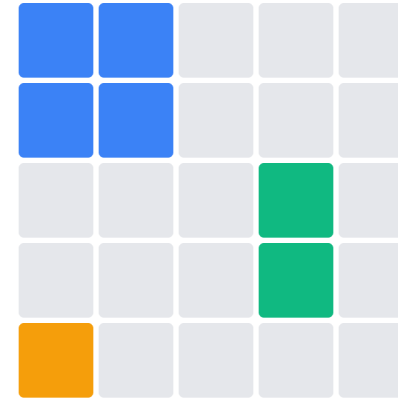
Dada uma **matriz binaria** (0 = fundo, 1 = primeiro plano),
contar quantos **objetos** existem usando **conectividade 8**.

Cada objeto e um grupo de celulas com valor 1 conectadas
horizontal, vertical ou diagonalmente.

Requisitos

ANSI C (C89/C90) | Pthreads | Resultados determinísticos

Exemplo 5x5 - 3 objetos



■ Objeto 1 ■ Objeto 2 ■ Objeto 3

Versao sequencial - BFS iterativa

Algoritmo

Percorre a matriz celula a celula. Ao encontrar um 1 nao visitado, inicia uma BFS iterativa marcando todo o componente.

Por que BFS iterativa?

Evita stack overflow em componentes grandes. Usa fila explicita ao inves de recursao.

Pseudocodigo

```
para cada celula (i, j):  
  se matriz[i][j] = 1  
  e nao visitado:  
    BFS(i, j)  
  objetos++
```

Complexidade: $O(L \times C)$
8 vizinhos por celula

PARALELISMO

Arquitetura paralela

Modelo

Pthreads com decomposicao por faixas de linhas (row-stripe)

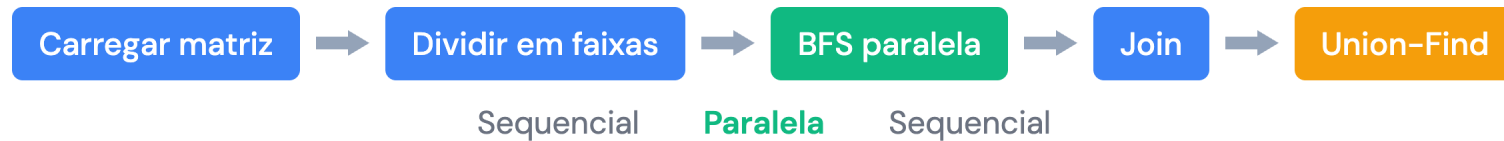
Trabalhadores

Configuravel via argumento de linha de comando

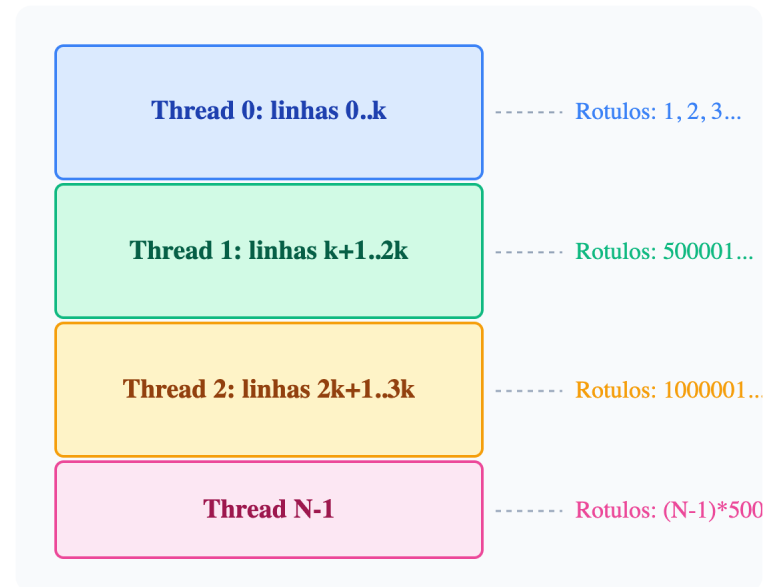
Sincronizacao

pthread_join como barreira implicita. Nenhum mutex necessario.

Fluxo de execucao



Divisao por faixas de linhas



Divisao estatica

Cada thread recebe L/N linhas. As primeiras $L\%N$ threads recebem uma linha extra.

Rotulos unicos

$\text{base_rotulo} = t * 500000$. Rotulos de threads diferentes nunca colidem.

Sem sincronizacao

Faixas disjuntas garantem ausencia de conflito na escrita.

CONSOLIDACAO

Union-Find nas fronteiras

Apos a BFS paralela, objetos que cruzam fronteiras entre faixas possuem **rotulos diferentes**. A consolidacao percorre cada fronteira e unifica rotulos equivalentes.

Direcoes verificadas na fronteira

S (abaixo)

SW (diagonal)

SE (diagonal)

Otimizacoes do Union-Find

- ✓ Compressao de caminho (path splitting)
- ✓ Uniao por rank
- ✓ Executado apos pthread_join (sem locks)

Exemplo rastreavel

```
rotulo[5][5] = 1 (Thread 0)
rotulo[6][6] = 500001 (Thread 1)
uf_union(1, 500001)
Resultado: mesmo componente
```

Sincronizacao e correcao

Sem condicoes de corrida

Threads escrevem em faixas disjuntas da matriz de rotulos. Sem sobreposicao = sem conflito.

Sem deadlocks

Nenhum mutex ou lock no programa. Unica sincronizacao: barreira implicita do pthread_join.

Deterministico

Resultados identicos entre sequencial e paralelo em todas as configuracoes testadas.

Recurso	Risco	Protecao	Justificativa
matriz[][]	Nenhum	Nenhuma	Somente leitura
rotulo[][]	Escrita concorrente	Particionamento	Faixas disjuntas
Union-Find	Corrida potencial	Fase sequencial	Apos pthread_join
Fila BFS	Nenhum	Alocacao local	Privada por thread

TESTES

Validacao funcional

5 matrizes obrigatorias – Todas aprovadas

Exemplo	Dimensoes	Esperado	Seq	Par
1	5 x 5	3	3	3
2	6 x 8	4	4	4
3	8 x 8	5	5	5
4	9 x 12	6	6	6
5	12 x 12	7	7	7

Matrizes adicionais

Dimensoes	Objetos	Threads	Status
100 x 100	461	1-8	OK
500 x 500	11.947	1-8	OK
1000 x 1000	47.911	1-8	OK
2000 x 2000	189.564	1-8	OK

5 repeticoes por configuracao. Resultado sempre identico entre versoes.

Metodologia experimental

Ambiente

Apple M4 (4P + 6E)
10 núcleos | 16 GB RAM
macOS Sequoia 15.7.3 | arm64

Medição

clock_gettime(CLOCK_MONOTONIC)
Trecho: threads + BFS + join + consolidação
Exclui leitura do arquivo

Matrizes testadas

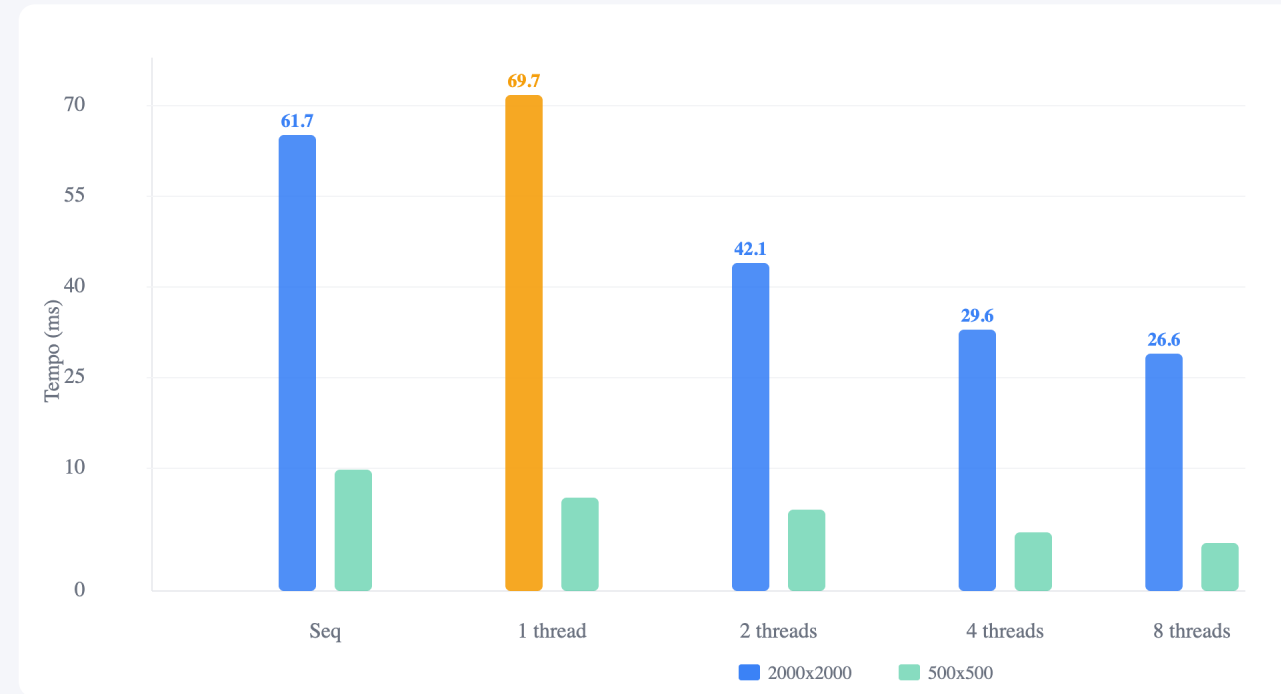
100x100 | 500x500 | 1000x1000 | 2000x2000
Densidade: 30% | Semente: 42

Configuração

Threads: 1, 2, 4, 8
5 repetições por configuração
Medida: média aritmética

DESEMPENHO

Tempo de execucao



2000x2000

61.7 ms

sequencial

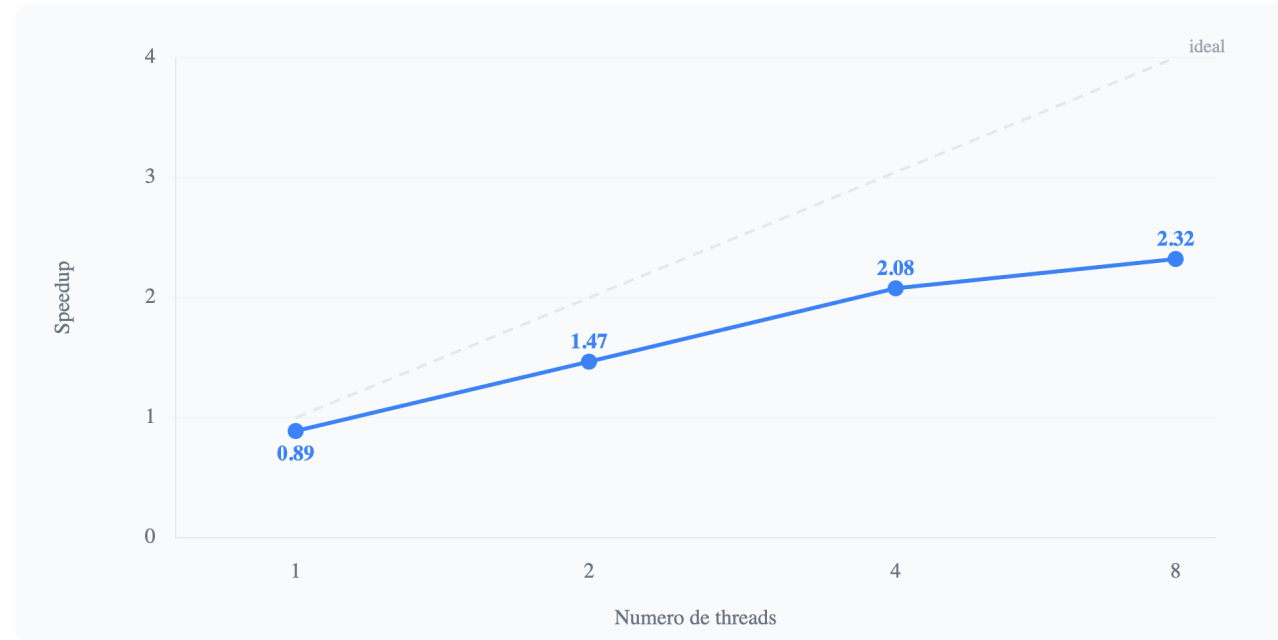
8 threads

26.6 ms

-57% tempo

ACELERACAO

$$\text{Speedup } S(p) = T_{\text{seq}} / T_{\text{par}}(p)$$



Melhor speedup

2.32x

8 threads, 2000x2000

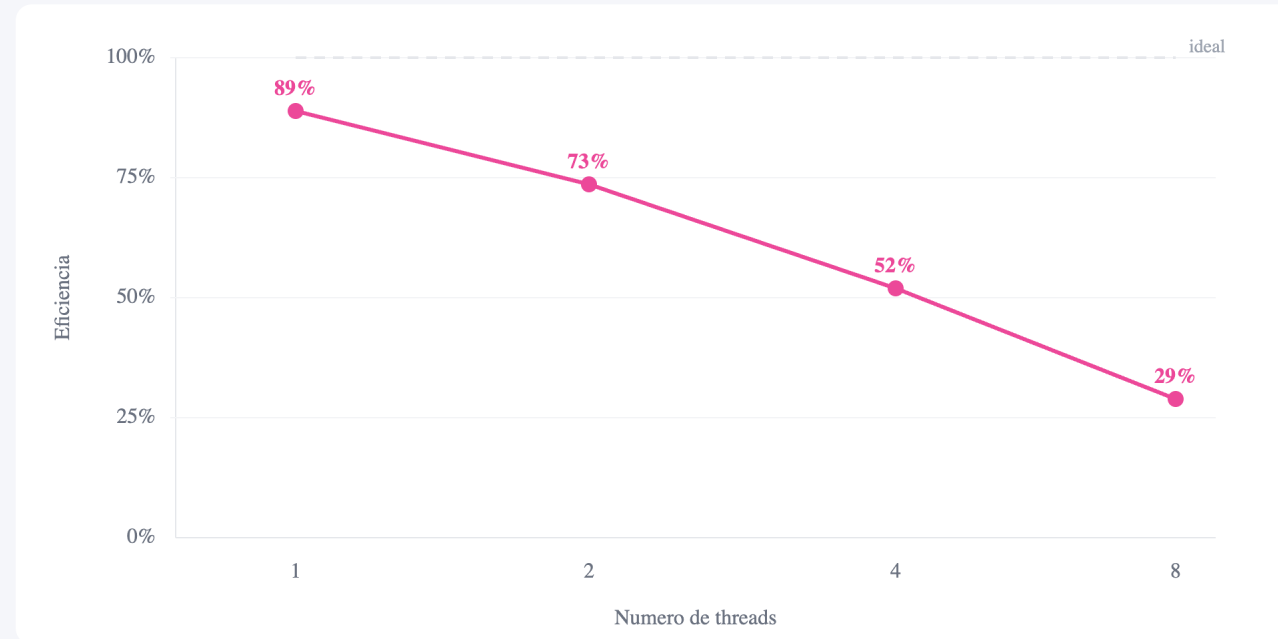
1 thread

0.89x

overhead de threads e
Union-Find

EFICIENCIA

$$\text{Eficiencia } E(p) = S(p) / p$$



Melhor eficiencia

89%

1 thread (overhead minimo)

8 threads

29%

Custos de gestao dominam

Eficiencia decresce com mais threads: overhead de criacao e consolidacao cresce mais rapido que o ganho.

Analise de desempenho

[icon: AlertTriangle] Overhead com 1 thread

Speedup 0.89x indica que a infraestrutura de threads (pthread_create, Union-Find, consolidacao) custa ~11% sobre o sequencial puro.

[icon: Maximize] Efeito do tamanho

Matrizes maiores se beneficiam mais do paralelismo. Em 500x500, os tempos sao tao curtos que o overhead domina. O ganho real aparece a partir de 1000x1000.

[icon: TrendingUp] Lei de Amdahl

A fase sequencial (Union-Find + consolidacao) limita o speedup maximo teorico. Com fracao serial estimada em ~15%, o limite seria ~6.7x.

[icon: Cpu] Arquitetura do M4

Apple M4 tem 4 nucleos P + 6 nucleos E. Com 8 threads, os nucleos E (mais lentos) participam, reduzindo a eficiencia por thread.

2-4

threads ideais

52-73%

faixa de eficiencia

~15%

fracao serial estimada

CONCLUSOES

Resultados e aprendizados

[icon: Zap] Desempenho

Speedup de 2.32x com 8 threads na matriz 2000x2000. Reducao de 57% no tempo de execucao (61.7ms para 26.6ms).

✓ Correcao

100% dos testes aprovados. Resultados identicos entre sequencial e paralelo em todas as configuracoes. Sem race conditions ou deadlocks.

[icon: BookOpen] Aprendizados

Programacao concorrente com Pthreads, estrategias de decomposicao de dados e estrutura Union-Find para consolidacao.

Trabalho futuro

Explorar balanceamento dinamico de carga, Union-Find lock-free com operacoes atomicas, e suporte a matrizes esparsas para melhorar escalabilidade.

2.32x
melhor speedup

[icon: MessageCircle]

Perguntas?

Obrigada pela atencao! Estamos a disposicao para
perguntas e discussao.

Elisa Rigotti & Maria Julia Escobar Correia de Melo

Sistemas Operacionais – PUCRS 2025/2